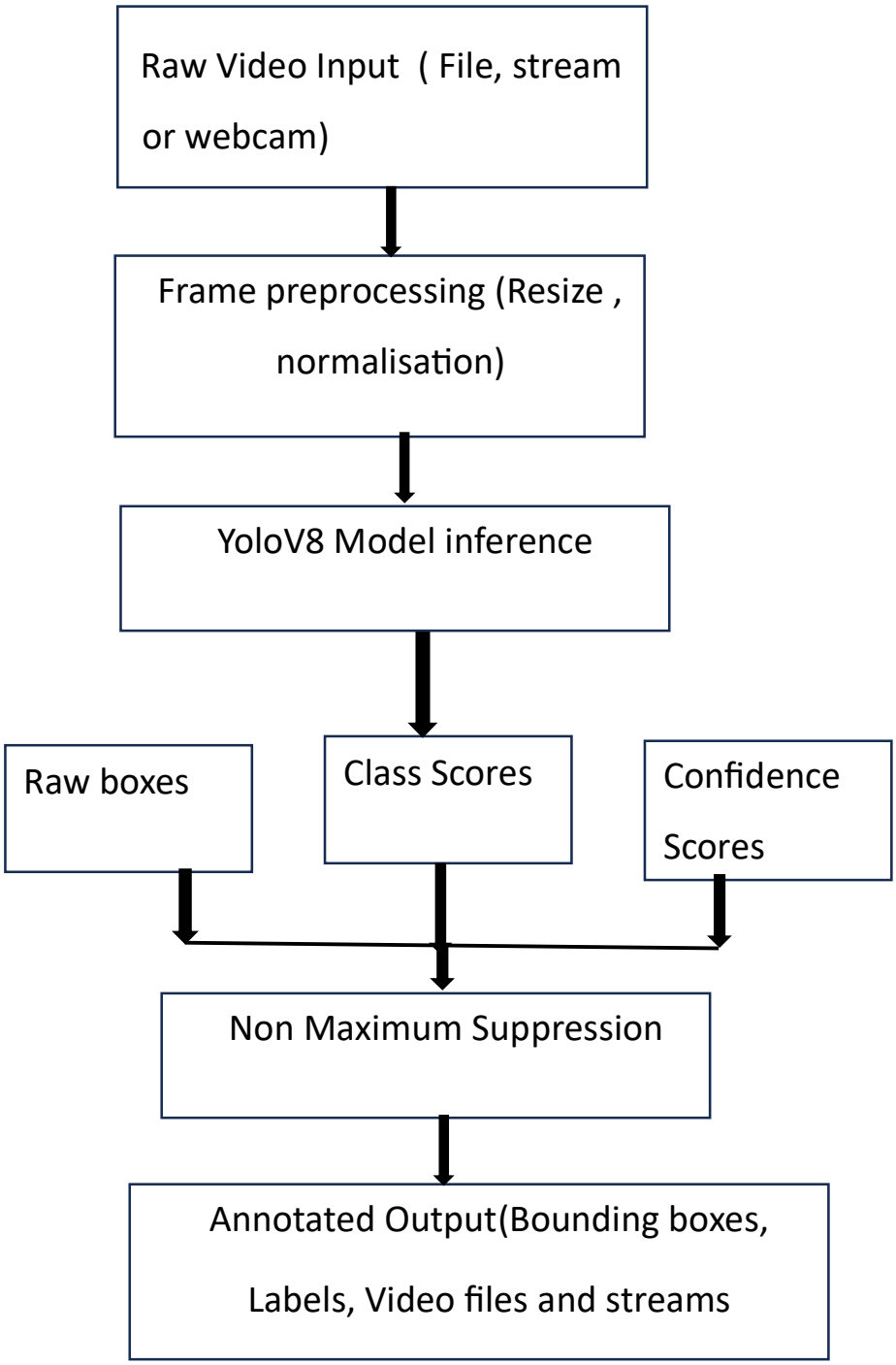
Edge Intelligence Laboratory- Object Detection pipeline(Yolo)

Oviya R (25MML0007)



1. Input

- Raw video is taken from:
  - File / Stream / Webcam

2. Frame Extraction

- Video is split into **individual frames**

3. Preprocessing

- Resize frame
- Normalize pixel values
- Convert image → tensor format

4. YOLOv8 Model Inference

- Model processes each frame
- Detects objects in the frame

5. Model Outputs

- **Raw Bounding Boxes** → object locations
- **Class Scores** → probability of each class
- **Confidence Scores** → detection accuracy

6. Feature Extraction Details

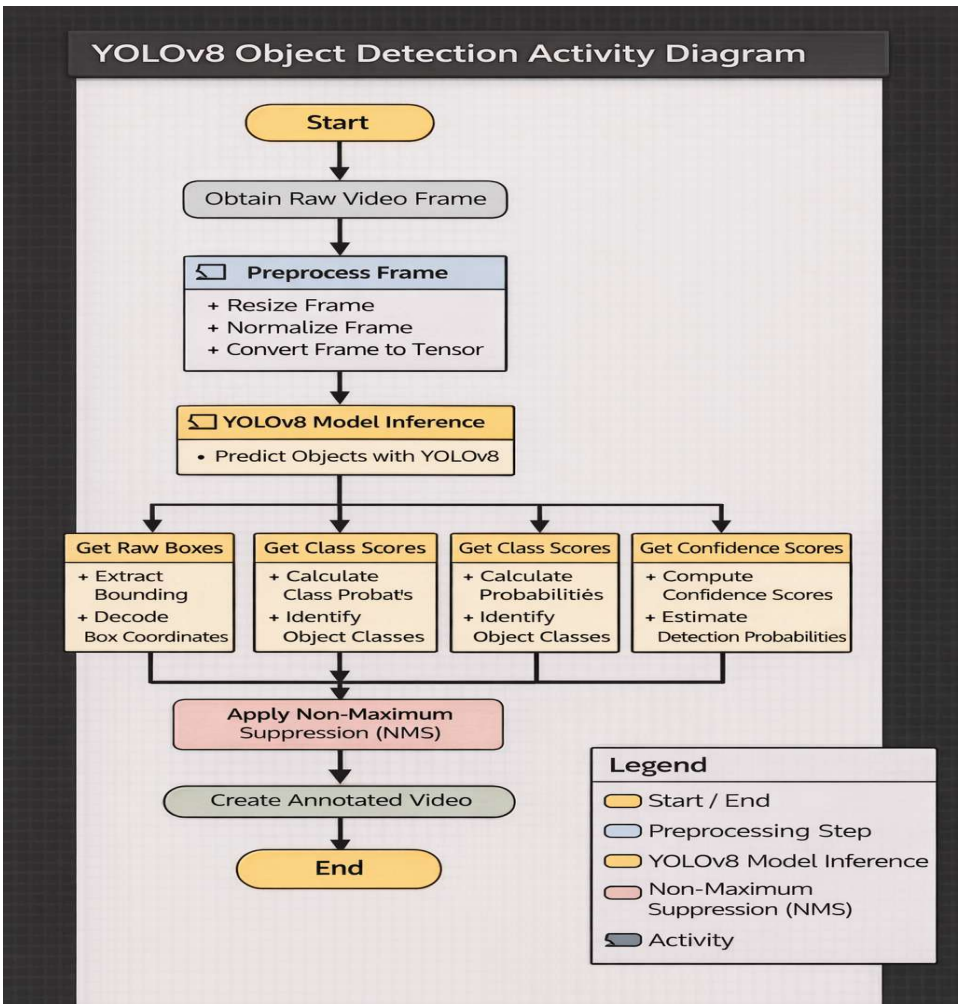
- **Get Raw Boxes**
  - Extract bounding boxes
  - Decode coordinates
- **Get Class Scores**
  - Calculate probabilities
  - Identify object class
- **Get Confidence Scores**
  - Compute confidence values
  - Estimate detection reliability

7. Non-Maximum Suppression (NMS)

- Removes duplicate/overlapping boxes
- Keeps best detection

8. Final Output

- Annotated video with:
  - Bounding boxes
  - Labels
  - Confidence scores



Flow:

Input image is given

1. Image is preprocessed (resized, normalized)
2. YOLO model predicts objects
3. Non-Max Suppression removes duplicate boxes
4. Final image is displayed

[1]:
```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
print("✅ Working!")
```

```
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\oviya\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultr
alytics-settings.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% ━━━━━━━━━━━━ 6.2MB 1.2MB/s 5.2s.2s<0.0s5s
3s
✅ Working!
```

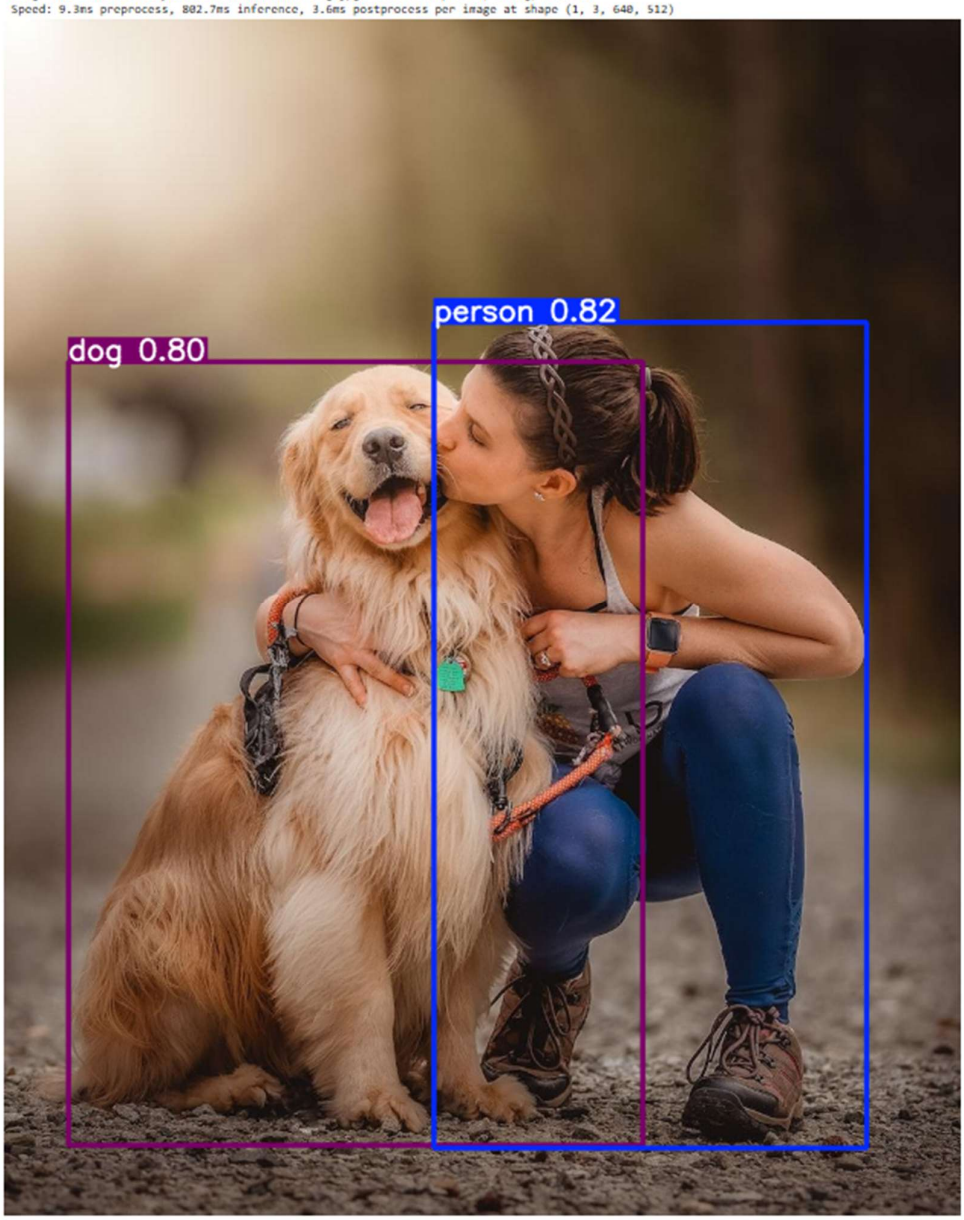
[4]:
```
from ultralytics import YOLO
from IPython.display import display
from PIL import Image

# Load model
model = YOLO("yolov8n.pt")

# Run detection
results = model(r"C:\Users\oviya\Downloads\humanand dog.jpg")

# Show result
img = Image.fromarray(results[0].plot()[:, :, ::-1])
display(img)
```

Output

- Image with:
    - Bounding boxes
    - Class labels (e.g., person, car)
    - Confidence scores

```
image 1/1 C:\Users\oviya\Downloads\humanand dog.jpg: 640x512 1 person, 1 dog, 802.7ms
Speed: 9.3ms preprocess, 802.7ms inference, 3.6ms postprocess per image at shape (1, 3, 640, 512)
```